EAST WEST UNIVERSITY

Project Title: Offline Multimodal Agentic RAG for Low-Resource Bilingual Agricultural Decision Support

Capstone Project Report- Spring 26

Course Code: CSE400B

Department of Computer Science & Engineering

Supervised By: Dr. Farhana Huq

Assistant Professor

Department of Computer Science & Engineering

Name	Student ID
Md. Asif Hossain	2022-3-60-007
Nabil Subhan	2022-3-60-063
Mantasha Rahman Mahi	2022-3-60-194
Jannatul Ferdous Nabila	2022-3-60-198

Date of Submission: 5.4.2026

Table of Contents

Table of Contents **2**
Chapter 1: Introduction **3**
1.1 Motivation 3
1.2 Problem Statement 3
1.3 Objectives 4
Chapter 2: Literature Review **4**
2.1 Background 4
2.2 Related Works 5
2.3 Research Gap: 6
Chapter 3: Research Questions & Objectives **7**
3.1 Research Questions 7
3.2 Objectives: 8
Chapter 4: Problem Statement and Requirement Engineering **9**
4.1 Problem Statement, Risk Assessment, and Objectives 9
4.2 Functional and Non-Functional Requirements 9
Functional Requirements 9
Non-Functional Requirements 10
4.3 System Constraints 11
Chapter 5: Methodology **11**
5.1 Methodology Overview 11
5.2 Data Sources and Collection 12
5.3 Data Preprocessing 13
5.4 Chunking and Metadata Preparation 13
5.5 Index Construction 14
5.6 Runtime Retrieval Pipeline 15
5.7 Agentic Answering, Verification, and Safety Controls 17
5.8 Local Large Language Model (LLM) Integration 18
Chapter 6: System Design and Implementation **18**
6.1 System Architecture Overview 18
6.2 Technology Stack and Framework 20
6.3 Prototype Implementation and User Interface 20
Chapter 7: Conclusion and Future Work **23**
7.1 Conclusion 23
7.2 Limitations of the Current Prototype 23
7.3 Future Scope and Implementation Plan 23
References **24**

Chapter 1: Introduction

1.1 Motivation

Bangladesh is an agriculture-based country, and many farmers depend on correct decisions about crop disease control, pesticide use, and fertilizer management. However, in rural areas internet connection is not always available or stable. Because of this, online solutions like cloud-based chatbots or web search may not work when people need help quickly. Also, many trusted agricultural resources are available as PDF manuals, but those documents are hard to search. They often contain tables, images, scanned pages, and repeated headers/footers, which makes it difficult to find the right information in short time. Because of this lack of reliable internet, it is a strict requirement for the system to run entirely locally on edge hardware. Furthermore, the complex nature of these agricultural PDFs necessitates a robust data ingestion pipeline (using tools like Marker and Tesseract OCR) to extract and chunk text accurately for retrieval.
Another big issue is language and dialect. Farmers usually describe problems in local Bengali words and dialect terms, while the manuals use formal Bengali, English, or scientific names. So even if the correct answer exists in a manual, a normal search system may fail to retrieve it using the farmer's natural query. To bridge this terminology gap, the integration of an offline SQLite Dialect Knowledge Graph is essential. By mapping local colloquialisms to formal scientific terms before searching, the system ensures high retrieval recall even when the user and the manual use completely different words.
In recent years, large language models can generate answers in a conversational way. But a normal LLM can sometimes guess or hallucinate information. In agriculture, wrong advice can cause loss of crops, money, and even health risks. For this reason, this project focuses on evidence-based answers with citations and safe behavior when information is missing.
These problems motivate building an offline-first decision support system that can work with limited hardware, support Bengali and English, accept voice and image input, and provide reliable answers from trusted documents. Given the high risks associated with agricultural misinformation, utilizing an Agentic Retrieval-Augmented Generation (RAG) approach rather than a standard LLM becomes imperative. This agentic workflow forces the model to generate answers only from retrieved evidence, provide exact page citations, and refuse to answer if the required context (like a specific chemical dosage) is missing.

1.2 Problem Statement

Farmers and agricultural extension officers in Bangladesh need a trustworthy assistant that works without internet and can provide correct guidance based on reliable documents. The system must support practical field constraints such as:

1. **Offline operation:** it should work fully offline after setup.
2. **Document complexity:** information is stored in PDF manuals with tables, images, and sometimes scanned pages.

3. **Language and dialect:** users may ask questions using dialect Bengali terms that do not match manual terminology.
4. **Safety and trust:** the system should not provide risky guesses (for example chemical dosage) if evidence is not found.
5. **Edge hardware limits:** the system should run on consumer hardware with limited GPU memory, so careful CPU/GPU resource usage is required.

1.3 Objectives

The main objective of this capstone project is to design and implement AgriBot, a production-grade offline multimodal Agentic RAG system for bilingual agricultural decision support. The key objectives are:

1. Build an offline ingestion pipeline to process agricultural PDFs and create searchable knowledge artifacts with page-level citations.
2. Implement a Retrieval-Augmented Generation (RAG) approach so that answers are generated based on retrieved evidence, not only model memory.
3. Use a Dialect Knowledge Graph to map local/dialect terms to canonical Bengali/English/scientific terms for better query understanding and retrieval.
4. Support multimodal inputs:
    - Text query (Bengali/English)
    - Voice query (Bengali speech-to-text)
    - Image input (crop/leaf photo analysis with OCR + symptom hints, and optional lightweight classifier)
5. Provide bilingual outputs (Bengali and English) while keeping citations unchanged and traceable.
6. Add an agentic workflow (self-correction and follow-up questions) so that the system can ask for missing context when confidence is low or evidence is not enough.
7. Provide a usable application through an API backend and a frontend interface suitable for offline kiosk-style usage.

Chapter 2: Literature Review

2.1 Background

Offline decision support systems are very important for rural environments where internet connection is not stable. In agriculture, farmers and extension officers often need fast guidance about crop diseases, fertilizer use, and pesticide dosage. However, most reliable information exists in manuals and extension documents, which are commonly shared as PDF files. These PDFs are difficult to search because they may contain tables, repeated headers, scanned pages, and images. As a result, users may fail to find the correct section even when the information is available.

Large Language Models (LLMs) can answer questions in natural language, but they may also generate incorrect information if they do not rely on trusted sources. To reduce this issue, Retrieval-Augmented Generation (RAG) is widely used. In a RAG system, the model first

retrieves relevant passages from documents and then generates the answer based on that evidence. This approach can improve reliability and also allows the system to provide citations, which is important for user trust in safety-sensitive domains like agriculture.
Another major challenge is language mismatch. Farmers may use local Bengali dialect terms, while manuals use formal Bengali, English, or scientific names. This mismatch can reduce retrieval performance. Knowledge-based methods such as entity linking and query expansion are commonly used to map informal terms to canonical terms and improve recall during retrieval.
Finally, practical deployment requires handling multimodal inputs. Voice-based queries are useful for users who prefer speaking, but speech-to-text errors can directly affect the quality of retrieval and the final answer. For this reason, many systems use confidence-based checks before generating a final response. Similarly, image inputs can provide symptom hints, but heavy vision-language models are difficult to run on low-VRAM devices. Lightweight CNN-based classification and rule-based visual heuristics are often used for edge-friendly vision support.

Overall, the background literature suggests that a trustworthy offline assistant should combine evidence-based answering (RAG), strong retrieval techniques, language/dialect alignment, and safety-focused interaction design.

2.2 Related Works

We have critically analyzed several key studies that directly relate to the components of our proposed solution. A summary of these works is presented in Table 2.1.

Table 2.1: Related Works

Paper Title & Author	Methodology & Focus	Limitations
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)	Introduces RAG: combines retrieval from a document index with generation, improving factuality and enabling provenance/citation-style answers.	Retrieval quality becomes a bottleneck; if wrong/weak evidence is retrieved, generation can still be incorrect. Also, typical setups assume strong infrastructure; offline deployment and safety policies are not the main focus. (nlp.cs.ucl.ac.uk)

5

The Probabilistic Relevance Framework: BM25 and Beyond (Robertson & Zaragoza, 2009)	Formal foundation for BM25, a highly effective sparse retrieval method for keyword-based document ranking	Sparse retrieval struggles when user vocabulary does not match the document vocabulary (e.g., Bengali dialect vs scientific terms). It also needs careful handling of noisy documents like PDFs with repeated headers/footers. (nowpublishers.com)

		allocation).
Billion-scale Similarity Search with GPUs (Johnson, Douze & Jégou, 2017)	Presents scalable vector similarity search methods; widely associated with FAISS-style indexing for dense retrieval systems.	Dense retrieval requires good embeddings and can miss exact numeric/keyword matches (e.g., dosage). It also needs offline index building and storage management for practical deployment. (huggingface.co)
Passage Re-ranking with BERT (Nogueira & Cho, 2019)	Uses a BERT cross-encoder for reranking retrieved passages, improving precision (e.g., MRR gains on MS MARCO).	Cross-encoders are computationally heavier; offline systems must manage latency and resource usage carefully. Reranking improves evidence selection but still depends on having good candidate passages. (huggingface.co)
Performant on-device inferencing with ONNX Runtime (Microsoft Open Source Blog, 2023)	Discusses ONNX Runtime for efficient on-device inference, relevant for CPU-bound model execution.	ONNX Runtime improves deployment feasibility but does not solve dataset/domain accuracy; a proper agriculture-trained classifier (labels + model artifact) is still required. opensource.microsoft.com

2.3 Research Gap:

The analysis of the literature shows that there is an important and still under-explored research gap for offline, safety-sensitive question answering systems in agriculture. Although Retrieval-Augmented Generation (RAG) has proven that grounded answering can reduce hallucination by generating responses from retrieved evidence, most existing RAG pipelines are designed mainly for general web-scale or cloud-based settings. In real rural deployment, reliable internet is not guaranteed, and the system must operate fully offline while still providing traceable citations from trusted manuals. This makes the problem harder because the system must manage local document storage, local indexes, and local inference under limited hardware. Another major gap is the language and terminology mismatch. Sparse retrieval methods like BM25 are strong baselines for keyword-based search, and dense vector retrieval has improved semantic matching at scale. However, both approaches can fail when the user uses colloquial Bengali and dialect terms that do not match the scientific terminology used in agricultural manuals. This mismatch is common in Bangladesh and can reduce retrieval recall even when the correct information exists in the documents. Therefore, there is a need for a robust query understanding approach that can align dialect terms to canonical entities and improve retrieval quality in low-resource language settings.

Evidence selection is also a critical issue. Reranking methods like BERT-based cross-encoders can improve precision by selecting more relevant passages, but they add extra computational cost and must be carefully designed for offline environments. Furthermore, even strong retrieval and reranking do not fully solve the need for safe interaction behavior. In agriculture, the system should not guess dosages or treatments when evidence is missing. Instead, it should verify evidence, refuse uncertain answers, or ask follow-up questions to gather missing context. This highlights a gap where a bounded, safety-aware agentic workflow is required beyond single-pass RAG.

Finally, multimodal input support creates additional challenges. Voice-based access is essential for many users, but speech recognition errors can directly affect retrieval and generation quality [6]. Similarly, image-based hints can be useful for crop diagnosis, but heavy vision-language models are difficult to run on low-VRAM edge hardware. This motivates a lightweight, CPU-friendly image analysis strategy that supports the retrieval process without making unsafe diagnostic claims.

Chapter 3: Research Questions & Objectives

3.1 Research Questions

Based on the identified gaps and the project objectives, we have formulated the following research questions:

RQ1: **How can an offline system provide reliable agricultural guidance by grounding answers in trusted PDF manuals and showing traceable citations?**

Answer: This project uses a Retrieval-Augmented Generation (RAG) approach where the system first retrieves relevant evidence from locally stored and pre-processed agricultural PDF documents, and then generates the final response based on that evidence. During ingestion, the documents are converted into chunks and stored with metadata such as document name and page information. At query time, the system uses the retrieved evidence to generate answers and can present citations so that users can verify the information. Since all indexes and documents are stored locally, the full pipeline works without the internet.

RQ2: How can retrieval performance be improved when users ask questions using colloquial Bengali and dialect terms that do not match the terminology used in formal agricultural documents?

Answer: This project introduces a Dialect Knowledge Graph (KG) that stores synonym and alias mappings between dialect terms and canonical agricultural terms (formal Bengali/English/scientific names). When a user asks a question, the system performs entity linking to identify important crop, disease, symptom, or chemical terms and then expands the query using KG relations and aliases. This improves recall because the retriever can match both the user’s original dialect words and the standardized terms found in manuals.

RQ3: How can an agentic workflow (self-correction and verification) reduce unsafe or overconfident answers by asking follow-up questions or refusing when evidence quality is low?
Answer: This project uses an agentic workflow that does not rely on a single retrieval attempt. The system can grade the quality of retrieved evidence, rewrite or expand the query, and retry retrieval within a bounded limit. If the system cannot find strong evidence, or if the user's question is incomplete (for example missing crop stage, dosage details, or location context), it will ask follow-up questions instead of generating a risky answer. This design reduces unsafe hallucinations and improves transparency.

RQ4: How effective and practical is multimodal interaction (voice and image input) in an offline setting, considering ASR errors, image quality issues, and limited hardware resources?
Answer: This project supports multimodal input with offline speech and image processing. For voice input, the system uses offline speech-to-text and applies confidence-based gating. If confidence is low, the system can request confirmation before continuing, which reduces errors that would otherwise harm retrieval. For image input, the system performs offline OCR and heuristic symptom extraction (for example yellowing, dark patches, brown spots). Optionally, a lightweight CPU-based classifier can provide top-k disease hints. The image outputs are treated as supportive signals for retrieval and follow-up questioning, not as a final clinical diagnosis. This makes multimodal interaction practical under offline and hardware constraints.

3.2 Objectives:

1. Build an offline ingestion pipeline to convert agricultural PDFs into searchable and citation-preserving knowledge artifacts.
2. Implement hybrid retrieval and evidence-based answer generation with citations.
3. Integrate a dialect knowledge graph to improve query understanding and retrieval recall.
4. Add an agentic self-correction and verification workflow that supports safe refusal and follow-up questions.
5. Support offline voice and image input pipelines with confidence/quality checks.
6. Provide a stable API backend and user-friendly frontend suitable for offline kiosk-style usage.

Chapter 4: Problem Statement and Requirement Engineering

4.1 Problem Statement, Risk Assessment, and Objectives

Bangladesh is an agriculture-based country, and a large number of farmers depend on correct and timely decisions for crop disease control, pesticide usage, and fertilizer management. However, in many rural areas the internet connection is unstable or unavailable. Because of this, online advisory systems and cloud-based AI tools cannot be depended on during critical field situations. At the same time, most trusted agricultural knowledge is available in PDF manuals and extension documents. These documents are difficult to search quickly because they often contain tables, scanned pages, images, and repeated headers/footers. As a result, farmers and extension officers may fail to find the right guidance even when it exists in official documents.
Another major challenge is the language and terminology gap. Farmers usually describe problems using local Bengali words and dialect terms, while agricultural manuals use formal Bengali, English, or scientific names. This mismatch makes information retrieval difficult. In practice, the user may ask a correct question, but the system may not retrieve the correct evidence because the words are different. This can lead to trial-and-error decisions or reliance on unverified advice. In agriculture, such mistakes can cause crop loss, financial damage, and unsafe chemical usage.
Risk Assessment and System Necessity:
Agriculture decisions are time-sensitive. If a disease spreads for a few days without the correct treatment, the damage can become severe and sometimes irreversible for that season. Wrong pesticide dosage or wrong treatment steps can also create health risks and increase production costs. Therefore, a reliable, offline, and evidence-based agricultural decision support tool is not only useful but necessary for rural communities where expert support is limited.
Primary Objectives:
The primary objective of this project is to develop AgriBot, a production-grade offline multimodal Agentic RAG system that can provide bilingual (Bengali and English) agricultural decision support by retrieving evidence from trusted PDF manuals, understanding dialect terms using a dialect knowledge graph, and supporting text, voice, and image-based queries. The system aims to produce safe and citation-backed answers, and to ask follow-up questions or refuse to answer when evidence is insufficient.

4.2 Functional and Non-Functional Requirements

Functional Requirements

1. **Offline Operation:** The system must work fully offline after setup. All documents, indexes, and models must be stored locally so that the system can answer without an internet connection.

2. **Document Ingestion and Indexing:** The system must accept trusted agricultural PDF documents and process them into a searchable knowledge base. This includes text extraction/OCR when needed, chunking, metadata storage, and index building for retrieval.
3. **Evidence Retrieval:** For each user query, the system must retrieve relevant passages from the local knowledge base and generate an answer based on the retrieved evidence instead of guessing.
4. **Citation Support:** The system must show citations for the answer such as document name, page number, or page reference so that users can verify the information from the original manual.
5. **Bilingual Support:** The system must support both Bengali and English queries and produce answers in Bengali and/or English based on user need.
6. **Dialect Handling using Knowledge Graph:** The system must support Bengali dialect terms by mapping them to canonical terms (formal Bengali/English/scientific terms) using a Dialect Knowledge Graph and query expansion.
7. **Agentic Safety Behavior:** The system must support safe interaction by doing bounded self-correction (e.g., retry retrieval with a rewritten query). If evidence is insufficient or the question is incomplete, the system must ask follow-up questions or refuse to answer rather than producing a risky response.
8. **Voice Input and Output:**
    a. The system must support offline speech-to-text for Bengali voice queries.
    b. The system must support offline text-to-speech so that answers can be played as voice output.
    c. If speech recognition confidence is low, the system should request confirmation of the transcript.
9. **Image-Based Query Support:** The system must accept crop or leaf images and extract useful hints. Optionally, a lightweight classifier may provide top-k condition hints. The image result should support retrieval and follow-up questions, not produce an overconfident diagnosis.
10. **User Interface and API:** The system must provide a usable interface and a backend API so that users can submit text, voice or image queries and receive answers with citations.

Non-Functional Requirements

1. **Reliability and Safety:** The system must prioritize grounded answers. When evidence is weak or missing, the system must provide cautious responses, ask follow-up questions, or refuse the request if it can be unsafe.
2. **Latency:** The system should respond within a reasonable time for offline usage. Text queries should return answers within a few seconds, and voice/image queries may take slightly longer but should remain usable for real field support.
3. **Offline Robustness:** The system must remain functional without the internet, and it must not crash due to common PDF issues such as scanned pages, repeated formatting, noisy text, etc.

4. **Resource Efficiency:** The system must run on consumer hardware with limited GPU memory. Heavy models should be avoided where possible, and CPU/GPU usage should be controlled to prevent out-of-memory errors.
5. **Scalability:** The architecture should support adding more PDF documents, more dialect mappings, and more users without major redesign. The indexing pipeline should allow updates when new documents are added.
6. **Usability:** The interface should be simple enough for non-technical users. The output should be easy to understand, and the system should guide the user using follow-up questions when needed.
7. **Privacy:** User inputs such as voice recordings and images should be processed locally, and the system should avoid sending data to external servers. Stored logs (if any) should be minimal and controlled.

4.3 System Constraints

This project is designed for offline and resource-constrained environments. Therefore, the following constraints are considered during design and implementation:

1. **Offline Constraint:** After initial setup, the system must operate without internet access. All documents, indexes, and models must be available locally.
2. **Hardware Constraint:** The system is intended to run on consumer-grade hardware with limited GPU memory and limited compute. This limits the use of heavy vision-language models and very large LLMs.
3. **Document Quality Constraint:** Many agricultural PDFs may contain scanned pages, noisy formatting, tables, and images. The ingestion pipeline must handle such document issues and still produce usable searchable content.
4. **Language/Dialect Constraint:** Users may use informal Bengali and regional dialect terms. The system must handle vocabulary mismatch through dialect mapping and query expansion.
5. **Safety Constraint:** Agricultural recommendations can be risky if the system guesses. The system must prefer citation-based answers and use follow-up questions or refusal when evidence is insufficient.
6. **Storage Constraint:** Local indexing and storing multiple documents require disk space management. The system must store indexes efficiently and allow new documents to be added when needed.

Chapter 5: Methodology

5.1 Methodology Overview

AgriBot follows an offline Agentic RAG methodology to provide bilingual agricultural decision support using trusted PDF manuals. At build-time, the system converts PDFs into structured evidence chunks, searchable indexes, and a dialect knowledge graph snapshot. At run-time, user inputs are normalized and enriched using KG linking and lightweight vision/ASR signals, then

hybrid retrieval and reranking select evidence for citation-backed answer generation with a bounded self-correction loop.

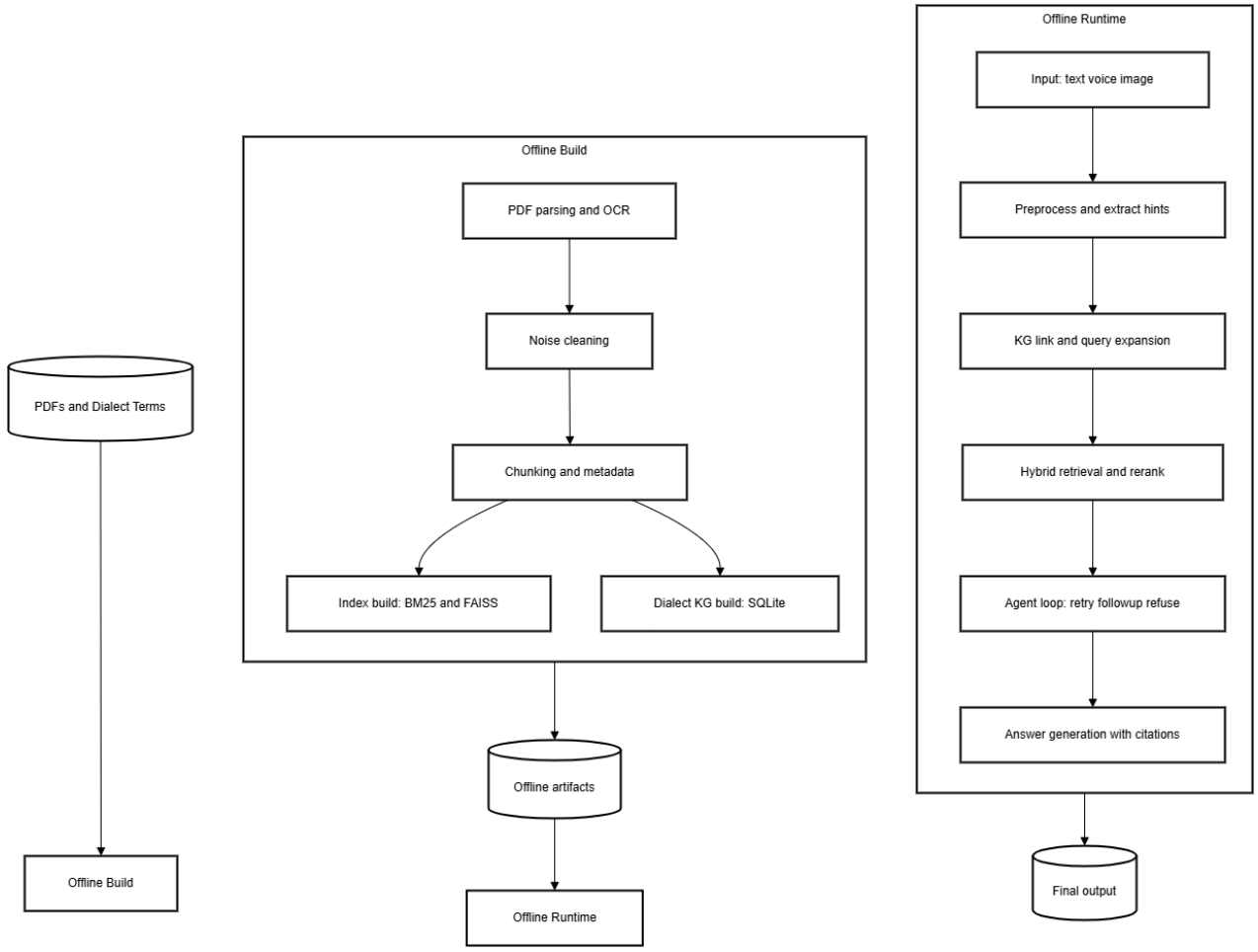

Figure 5.1: Methodology Flowchart of AgriBot

5.2 Data Sources and Collection

The data collection process prioritizes reliability and coverage of common crop disease and treatment scenarios.

Agricultural Document Data:
The primary knowledge source consists of agricultural PDF manuals and guideline documents for example crop disease management, pest control, fertilizer use, safe pesticide handling etc. These documents are selected based on (i) official research institute (ii) extension manuals, (iii) relevance to Bangladesh agriculture etc. All PDFs are stored locally and later processed in the ingestion pipeline to produce chunk-level evidence with page references for citation.

Dialect and Terminology Data:
To reduce the terminology gap between farmer language and formal agricultural manuals, AgriBot builds an offline Dialect Knowledge Graph that maps colloquial Bangla terms and spelling variants to canonical Bengali/English entities. In our project, the alias terms were collected from authentic BRRI (Bangladesh Rice Research Institute) PDF manuals, especially glossary-style sections where local names, alternate spellings, and related terms appear. We manually curated these terms into a structured knowledge base.

Vision Data:
To support image-based queries in AgriBot, we use a labeled Rice Leaf Diseases image public dataset available on kaggle for training a lightweight rice disease classifier intended for offline CPU deployment. The dataset is divided into Training, Validation, and Testing splits, and each split contains the same set of condition classes. Based on the dataset structure used in this project, the classes include Bacterial Leaf Blight, Brown Spot, Leaf Blast, Leaf Scald, Narrow Brown Leaf Spot, Rice Hispa, Sheath Blight, and Healthy Rice Leaf. All samples are provided as .jpg images and are used for supervised multi-class classification.

5.3 Data Preprocessing

1. **Input PDFs:** Agricultural manuals are stored locally and loaded from the configured PDF directory.
2. **Layout-aware PDF parsing:** AgriBot uses Marker to extract structured text while preserving reading order and handling complex layouts such as headings, multi-column text, tables, figures.
3. **OCR fallback:** If a page has very low extracted text after Marker, the system renders the page as an image with 300 DPI and applies Tesseract OCR. The OCR result replaces the existing text only if it contains more usable content.
4. **Header/footer noise removal:** Repeated lines that appear across a large fraction of pages within the same PDF are detected based on frequency and removed to prevent retrieval pollution.
5. **Non-content page handling:** Pages that look like table of contents, index or references, and pages with very low text, are detected using keyword + pattern rules and assigned a lower keep weight so they do not dominate retrieval.

5.4 Chunking and Metadata Preparation

1. **Purpose of chunking:** After preprocessing, each PDF page is split into smaller text segments also known as chunks so the retrieval system can search and cite specific evidence instead of entire pages.
2. **Chunking method:** AgriBot uses a Recursive Character Text Splitter to break page text into chunks based on separators such as paragraph breaks, newlines, sentences, spaces etc. while staying within a fixed character limit.

3. **Chunk size and overlap:** Chunking uses a configurable chunk size in our case 600 characters and an overlap of 120 characters to reduce context loss between consecutive chunks.
4. **Minimum chunk filtering:** Very short fragments are removed using a minimum length threshold to avoid indexing low-signal text.
5. **Provenance metadata stored per chunk:** Every chunk stores PDF filename, page number, global chunk index, content / toc / reference / low_signal page-level weight for downweighting low-value pages as metadata. So that this metadata is later used for citation generation.

5.5 Index Construction

AgriBot needs to retrieve relevant agricultural evidence without internet access. So it is necessary to convert the cleaned chunks into two complementary offline search indexes so AgriBot can retrieve relevant evidence without internet access.

1. **Dense indexing (FAISS)**:
    - Each chunk is converted into a dense vector using a SentenceTransformer embedding model.
    - FAISS IndexFlatIP stores these vectors and uses inner product on normalized vectors to find nearest neighbors.

2. **Sparse indexing (BM25)**:
    - Each chunk is also tokenized and indexed using BM25Okapi for exact-term matching which is useful for specific keywords, chemical names, and numbers.

Finally, save these two search indexes to the computer's disk so AgriBot can find relevant info even without the internet. The reason for using both is dense retrieval captures semantic similarity, while BM25 preserves exact keyword matching; together they improve recall and reduce missed evidence in technical agricultural queries.

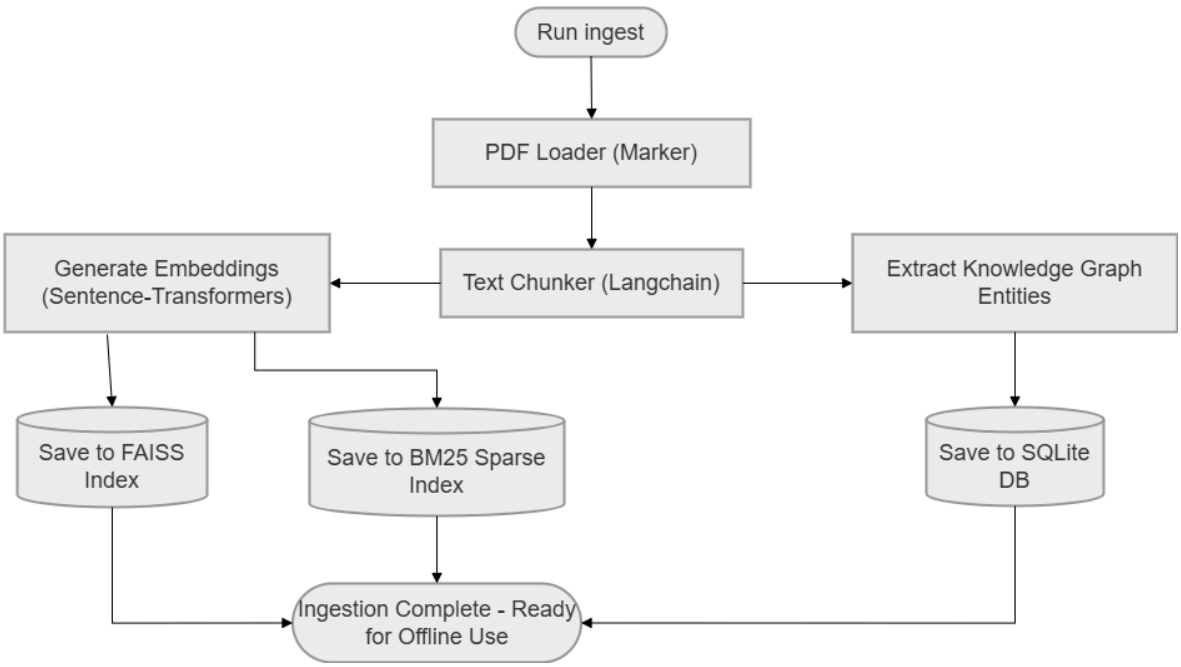

Figure 5.2: Data Ingestion and Offline Index Construction Pipeline

5.6 Runtime Retrieval Pipeline

At runtime, AgriBot must transform a farmer's query into a set of highly relevant, citation-ready evidence chunks before generating an answer. The query may be provided in Bangla or English and may also originate from voice transcription or an image-assisted request. It is first normalized to reduce noise and improve downstream matching consistency. After normalization, AgriBot performs Dialect Knowledge Graph (KG) entity linking to bridge colloquial terms to canonical agricultural terminology. Using the SQLite-backed KG, the system matches query tokens against stored aliases and expands the query with canonical Bengali/English terms and relevant variants. This query expansion step is critical for recall when user language differs from the formal wording found in manuals.

Once the query is expanded, AgriBot runs hybrid retrieval over the locally stored offline indexes created during ingestion. Dense semantic retrieval is performed by embedding the expanded query and searching the FAISS index for nearest-neighbor chunks, while sparse retrieval uses BM25 to capture exact keyword matches for instance disease names, chemical names, dosage-related terms etc. The candidate lists from dense and sparse retrieval are then merged using weighted Reciprocal Rank Fusion (RRF), and chunk keep_weight is applied so that low-value pages such as TOC, reference-like pages do not dominate the evidence set.
Next, the merged candidates are passed through a cross-encoder reranker to reorder and filter evidence more precisely. Only the top-N reranked chunks above a minimum score threshold are kept as the final grounding context. Finally, the system constructs a consolidated evidence context where each chunk is paired with its provenance label, ensuring that the subsequent answer-generation step can produce citation-backed outputs.

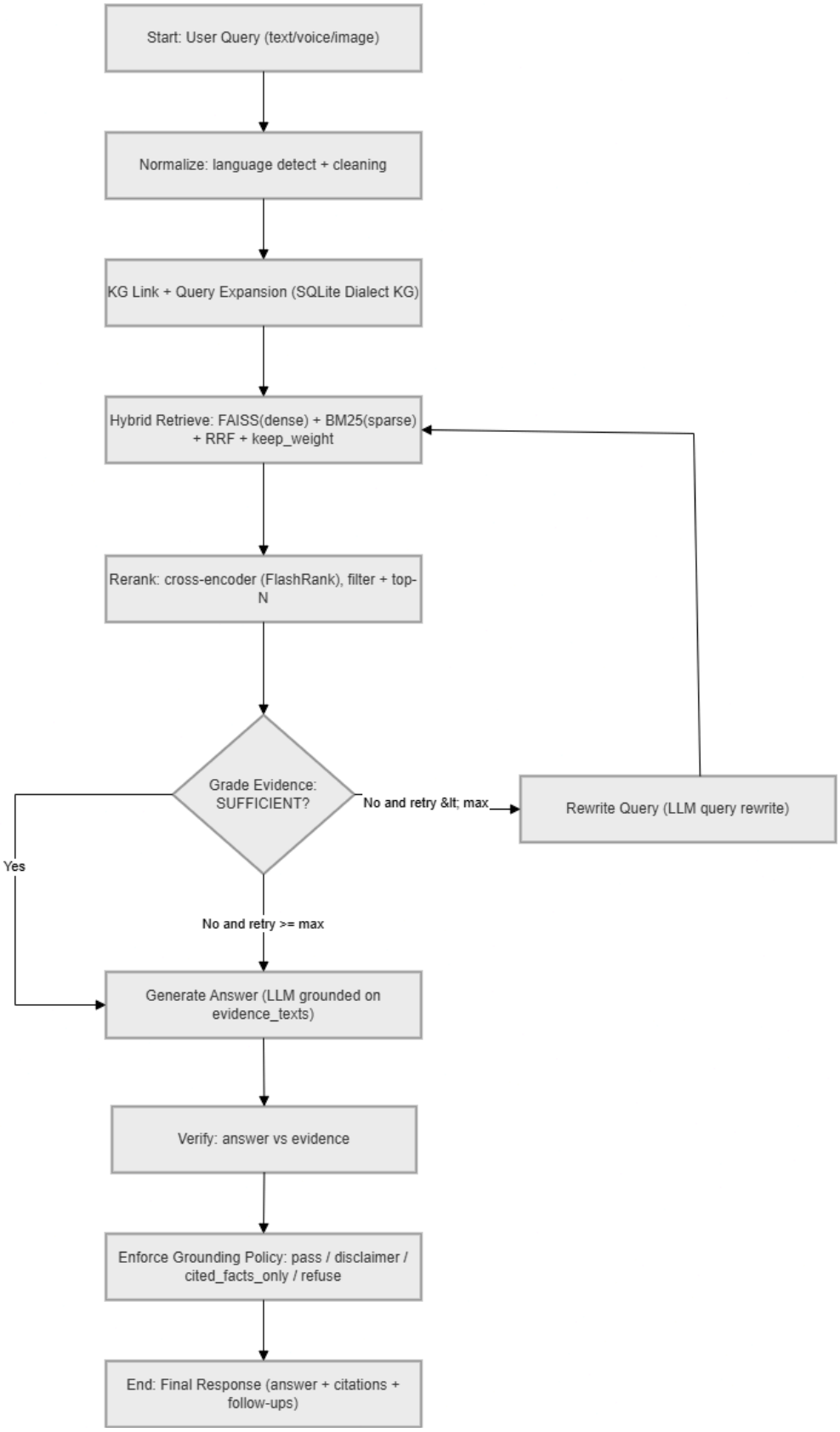

Figure 5.3: LangGraph Agent Workflow

5.7 Agentic Answering, Verification, and Safety Controls

After AgriBot completes retrieval and reranking, the system enters an agentic answering stage designed to ensure that the final response is grounded in retrieved evidence, verified, and safe for agricultural use. This stage is implemented as a LangGraph workflow where each node performs one well-defined step and writes outputs back into a shared agent state such as normalized query, evidence texts, citations, final answer, verification status, and policy decision etc. To support debugging and evaluation, each node also records its execution time which makes the pipeline observable and measurable.

First, AgriBot generates an answer using a local LLM and a strict evidence-grounded prompt. The generation prompt explicitly instructs the model to use only the provided evidence, to provide citations with file name and page number, and to refuse if the evidence does not support the requested information especially for sensitive topics such as dosage or chemical application rates. The evidence itself is passed as a structured context string where each paragraph is prefixed with its provenance label, enabling the model to cite sources naturally in the response. If the system previously determined that no evidence is available, it returns a safe “I don’t know based on the provided documents” response instead of guessing.

Next, the system supports bilingual usability through an optional translation step. Once an English answer is produced, AgriBot can translate the answer into Bengali using a dedicated translation component. BanglaT5 is used in this case. This step runs after generation and is skipped when the system is already refusing or when a Bengali answer is already present. The goal is to keep generation consistent , while still delivering a farmer-friendly Bangla output.

After generation , AgriBot runs a verification step that acts like an internal fact-checker. Using the same local LLM, the verifier compares the generated answer against the evidence context and outputs whether the answer is VERIFIED or UNVERIFIED with a reason. The verification is intentionally strict: dosage values, chemical names, and specific recommendations must appear in the retrieved evidence to be considered verified. If verification fails due to errors or uncertainty, the system conservatively marks the answer as unverified so that the next safety layer can decide how to respond.

Finally, AgriBot applies a grounding and safety policy layer that determines the final output behavior. This policy detects high-risk queries especially those involving numeric dosage units, pesticide/fungicide/herbicide application amounts, or Bengali dosage-related terms such as “মাত্রা”, “কীটনাশক” etc. and refuses to provide unverified advice. For non-risky queries, the policy can take one of several controlled actions depending on configuration: it may pass the answer as-is, append a disclaimer when verification is incomplete, restrict the response to “cited facts only” by extracting only evidence-supported sentences, or refuse completely when strict grounding is required. This ensures AgriBot remains helpful for general guidance while reducing harm from unsupported or hallucinated agricultural recommendations.

5.8 Local Large Language Model (LLM) Integration

To fulfill the strict offline deployment constraints, cloud based LLM APIs are entirely bypassed in favor of local edge-inference. AgriBot integrates the Qwen2.5-1.5B-Instruct model, deeply quantized into the Q8_0 GGUF format. This highly compressed model configuration drastically reduces the required RAM footprint, allowing inference to run efficiently on low resource, CPU-bound consumer hardware or edge devices without inducing Out Of Memory (OOM) errors.

The execution engine is powered by `llama-cpp-python, which optimizes token generation speeds for edge deployments. Additionally, the generation pipeline integrates structured decoding logic when strict formatting is demanded by the frontend application. By exclusively leveraging smaller, heavily quantized models constrained rigorously by the retrieved context, the architecture achieves a highly performant balance between linguistic fluency and computational efficiency.

Chapter 6: System Design and Implementation

6.1 System Architecture Overview

The architecture of AgriBot is designed as a centralized, offline-first system to manage multiple heavy computational tasks on low-resource hardware. As illustrated in Figure 6.1, the user interacts via the AgriBot UI, which forwards requests to the central FastAPI Backend. This backend acts as the core orchestrator. Depending on the type of input and the stage of the retrieval process, the backend asynchronously communicates with several localized components: the Offline Indexes (FAISS + BM25 + chunks metadata) for evidence retrieval, the SQLite Dialect KG for term mapping, the Local LLM for answer generation, and the Translator for bilingual output. Additionally, for multimodal inputs, the backend routes data to the Offline STT (faster-whisper), Offline TTS, or the Optional Image Pipeline. This centralized API design ensures that the UI remains responsive while the FastAPI backend handles the heavy, time-consuming inference tasks such as audio transcription and LLM generation in the background.

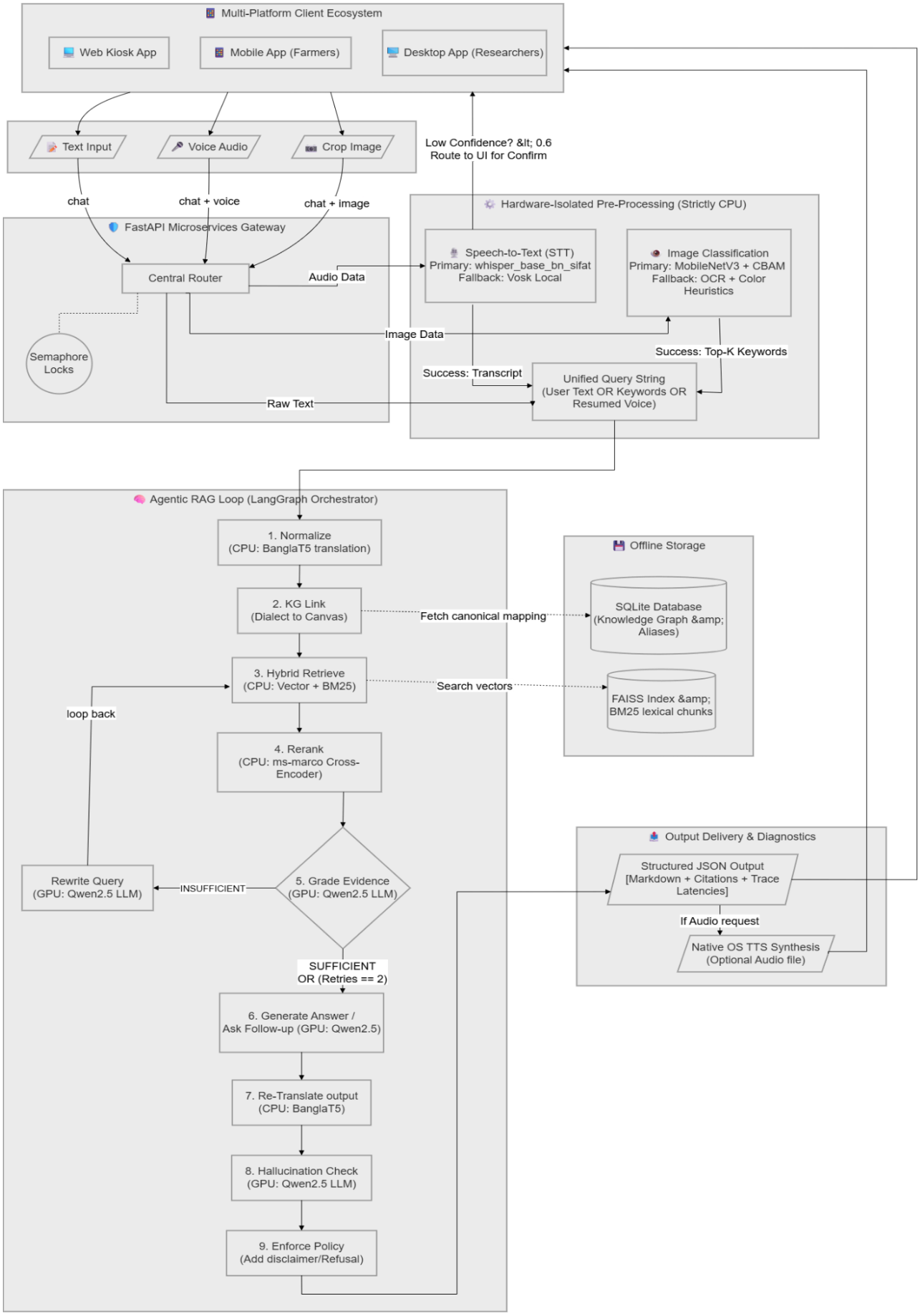
Multi-Platform Client Ecosystem
Web Kiosk App
Mobile App (Farmers)
Desktop App (Researchers)
Text Input
Voice Audio
Crop Image
Low Confidence? < 0.6
Route to UI for Confirm
chat
chat + voice
chat + image
FastAPI Microservices Gateway
Central Router
Audio Data
Semaphore Locks
Hardware-Isolated Pre-Processing (Strictly CPU)
Speech-to-Text (STT)
Primary: whisper_base_bn_sifat
Fallback: Vosk Local
Image Classification
Primary: MobileNetV3 + CBAM
Fallback: OCR + Color Heuristics
Image Data
Success: Top-K Keywords
Success: Transcript
Unified Query String
(User Text OR Keywords OR Resumed Voice)
Raw Text
Agentic RAG Loop (LangGraph Orchestrator)
1. Normalize
(CPU: BanglaT5 translation)
2. KG Link
(Dialect to Canvas)
Fetch canonical mapping
Offline Storage
SQLite Database
(Knowledge Graph & Aliases)
3. Hybrid Retrieve
(CPU: Vector + BM25)
Search vectors
FAISS Index & BM25 lexical chunks
loop back
4. Rerank
(CPU: ms-marco Cross-Encoder)
Output Delivery & Diagnostics
Structured JSON Output
[Markdown + Citations + Trace Latencies]
Rewrite Query
(GPU: Qwen2.5 LLM)
INSUFFICIENT
5. Grade Evidence
(GPU: Qwen2.5 LLM)
If Audio request
Native OS TTS Synthesis
(Optional Audio file)
SUFFICIENT
OR (Retries == 2)
6. Generate Answer /
Ask Follow-up (GPU: Qwen2.5)
7. Re-Translate output
(CPU: BanglaT5)
8. Hallucination Check
(GPU: Qwen2.5 LLM)
9. Enforce Policy
(Add disclaimer/Refusal)

Figure 6.1: High-Level System Architecture of AgriBot

6.2 Technology Stack and Framework

To fulfill the offline and low-resource constraints, the specific technologies depicted in the system architecture (Figure 6.1) have been carefully selected:

1. **Core API Orchestration (FastAPI Backend):** FastAPI provides high-performance, asynchronous request handling. Since offline inference models (LLM, STT, TTS) take significant time to process on CPU or low-end GPUs, asynchronous execution is mandatory to prevent system timeouts.
2. **Search and Terminology Mapping (Offline Indexes & SQLite Dialect KG):** Using FAISS for dense context matching and BM25 for exact keyword matching ensures maximum retrieval recall. The Dialect Knowledge Graph is stored in SQLite, providing a highly lightweight, serverless database solution to map local farmer dialects to formal agricultural terms without requiring heavy graph database software.
3. **Language Models (Local LLM & BanglaT5):** Running a full-precision LLM requires massive VRAM. By utilizing a quantized Local LLM, the memory footprint is drastically reduced. Furthermore, generating answers in English and explicitly routing them through the BanglaT5 Translator ensures higher linguistic accuracy than relying solely on a small LLM for direct Bengali generation.
4. **Multimodal Pipelines (faster-whisper, TTS, and Image Pipeline):** To support illiterate users or those in the field, faster-whisper is utilized for offline Speech-to-Text (STT) transcription, offering high accuracy with minimal compute overhead. The TTS component provides audio feedback. For image queries, a lightweight pipeline utilizing OCR and a CNN-based classifier extracts symptom hints without overloading edge hardware.

6.3 Prototype Implementation and User Interface

A functional prototype of the AgriBot system has been developed to validate the proposed offline architecture and demonstrate its multimodal capabilities. The presentation layer is explicitly designed to function as an interactive kiosk, prioritizing accessibility for rural farmers and agricultural extension officers who may have limited technical literacy.

1. **Primary Interface Design:** As depicted in Figure 6.2, the AgriBot UI offers a clean, distraction-free environment. The interface abstracts the complex backend processes (such as dialect mapping and hybrid retrieval) away from the user, presenting a simple chat-based interaction model.

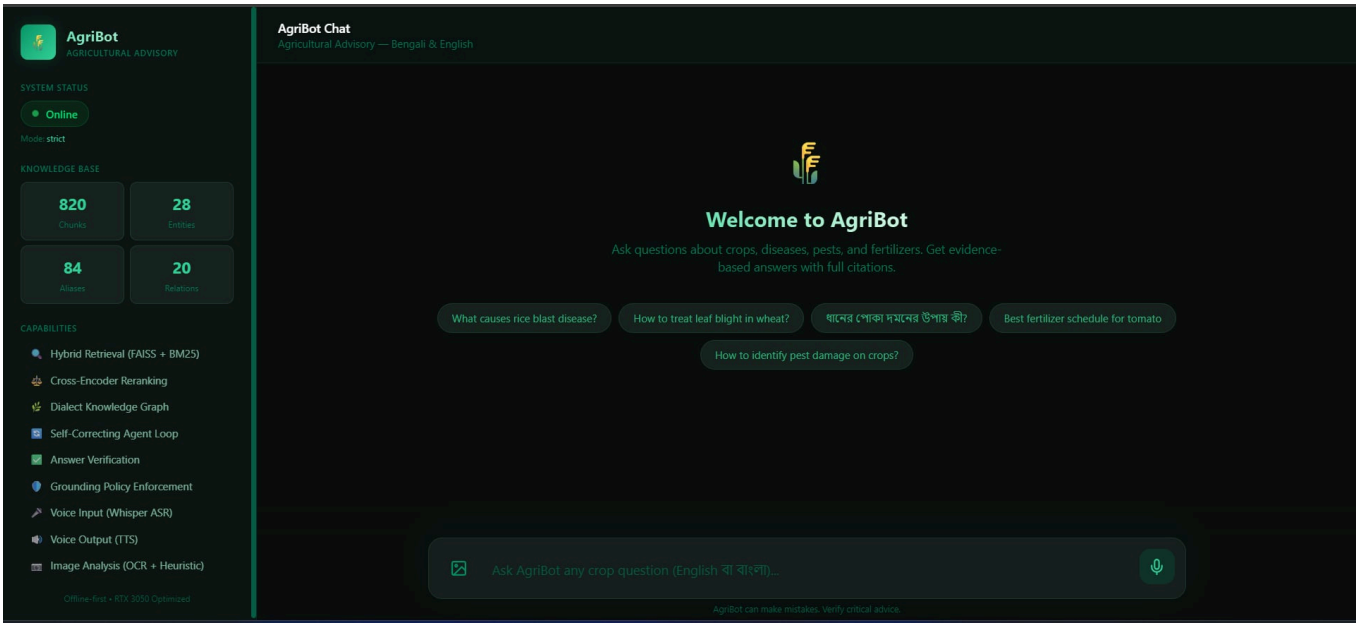

Figure 6.2: AgriBot UI

2. **Text-Based Interaction and Citation Display:** Figure 6.3 demonstrates the system's response to a standard text input. When a user submits a query, the system retrieves relevant evidence and generates a grounded response. Crucially, the UI is programmed to explicitly display the source citations for example, the specific PDF manual name and page number alongside the generated answer. In the agricultural domain, hallucinated advice regarding pesticide dosage or disease treatment can cause severe financial and physical harm. Displaying exact citations fulfills the system's safety and trustworthiness constraints, allowing users to manually verify the AI's advice against trusted documents.

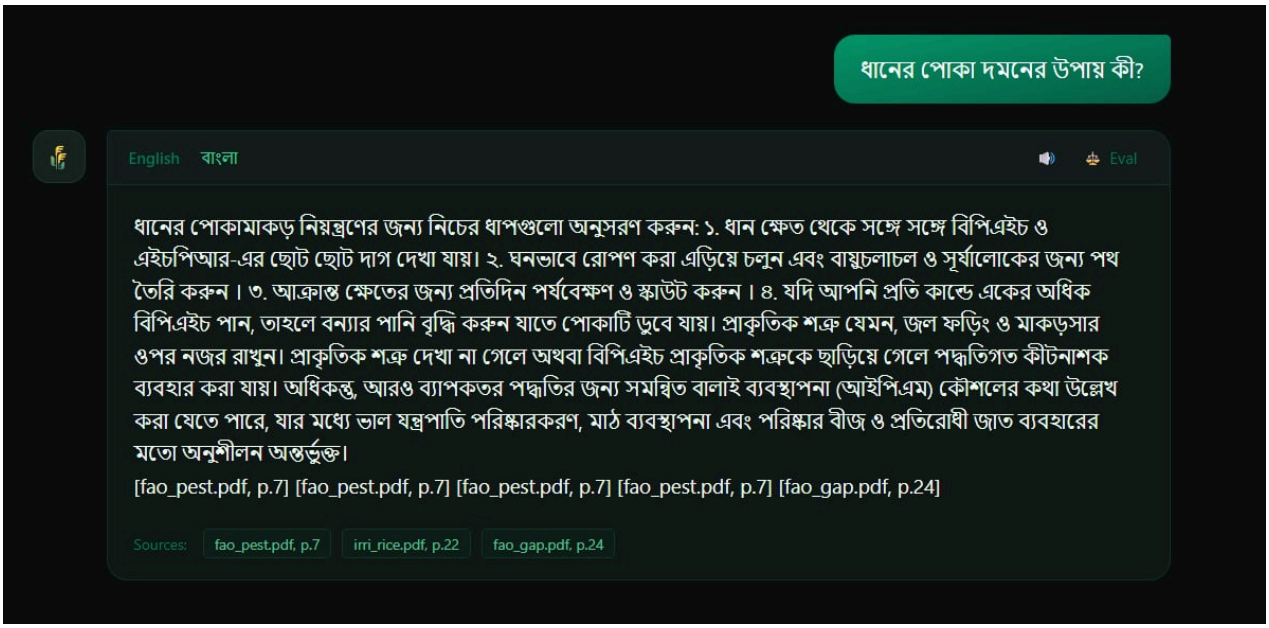

Figure 6.3: Q&A response with text input

3. **Multimodal Voice Interaction:** To accommodate users who are not comfortable typing in Bengali or English, the system integrates a seamless voice input mechanism, as illustrated in Figure 6.4. By utilizing the microphone interface, the user's spoken query is captured and processed locally via the offline faster-whisper Speech-to-Text (STT) module. The transcribed text is then fed into the standard retrieval pipeline. Text-only interfaces create a significant accessibility barrier in rural Bangladesh. The integration of offline voice recognition ensures the system is practically usable in field conditions, while processing the audio entirely on local hardware ensures data privacy and zero reliance on internet connectivity.

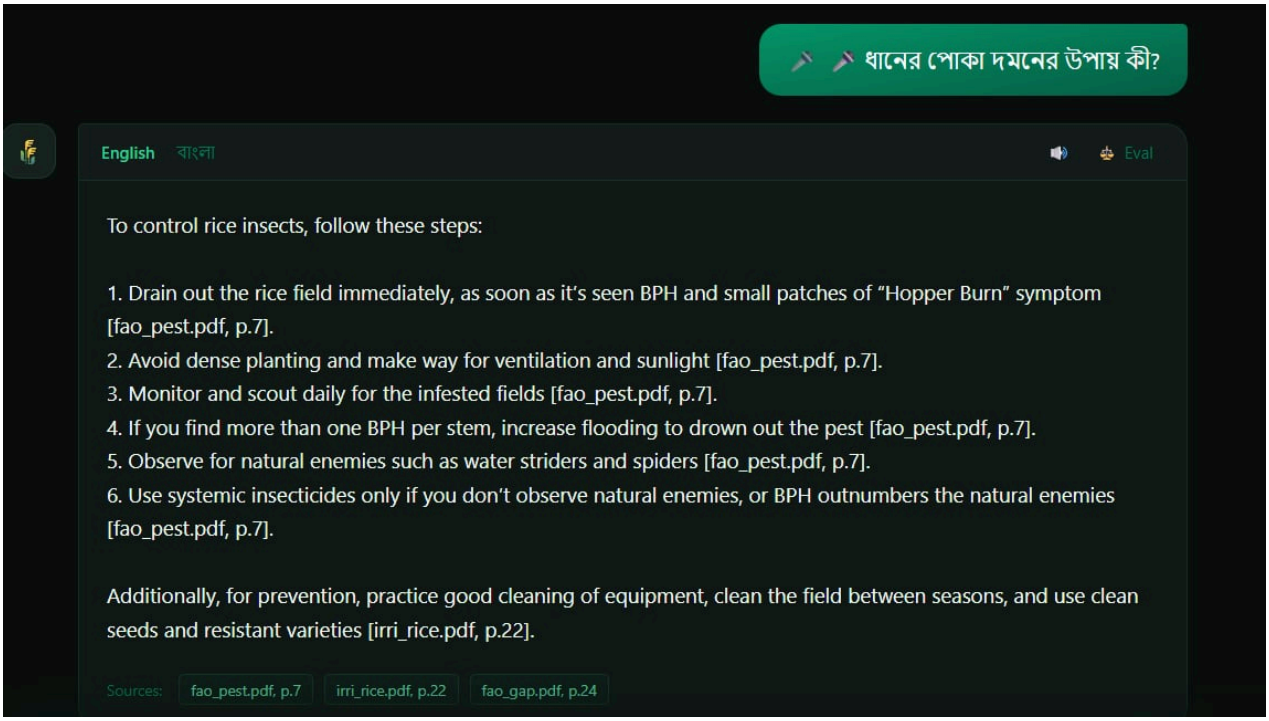

Figure 6.4: Q&A response with voice input

4. **Planned Image-Based Interaction Pipeline:** While the current prototype successfully demonstrates text and voice modalities, the architecture as outlined in Figure 6.1 is designed to accommodate image-based queries in the final implementation phase. The UI will be updated to include an image upload interface for crop and leaf photos. These images will be processed locally using a lightweight Convolutional Neural Network for symptom classification and Tesseract OCR for text extraction. Farmers often cannot describe visual symptoms accurately in text or voice. Allowing image uploads will provide the LangGraph agent with deterministic visual hints, significantly improving the retrieval accuracy for disease-specific manuals.

Chapter 7: Conclusion and Future Work

7.1 Conclusion

In an agriculture-dependent country like Bangladesh, timely and accurate decision support can prevent catastrophic crop losses. However, the reliance on internet-dependent cloud AI systems and the terminology gap between formal agricultural manuals and local dialects create significant barriers. This project proposes AgriBot, an offline, multimodal, Agentic RAG system tailored for low-resource environments. The completed prototype successfully establishes the core backend orchestration, the SQLite Dialect Knowledge Graph, and hybrid retrieval indexes. Furthermore, the frontend currently supports offline text and voice queries, providing citation-backed, bilingual guidance without compromising safety or relying on unstable internet connections.

7.2 Limitations of the Current Prototype

As this project is in the proposal and prototyping phase, the current iteration possesses certain limitations that will be addressed in the final build:

1. **Incomplete Vision Pipeline:** The multimodal capability is currently restricted to audio and text. The offline image classification model for leaf disease detection is pending integration.
2. **Hardware-Bound Latency:** Operating entirely offline using quantized LLMs and faster-whisper on consumer-grade hardware introduces higher response latency compared to cloud-based APIs.
3. **Knowledge Graph Coverage:** The current SQLite Dialect KG is populated with a limited subset of terms from BRRI manuals and requires expansion to cover a wider variety of regional Bangladeshi agricultural dialects.

7.3 Future Scope and Implementation Plan

The proposed architecture establishes a strong foundation. The primary objectives for the remainder of the capstone project timeline include:

1. **Integration of the Image Processing Pipeline:** A lightweight, CPU-friendly CNN model trained on the Kaggle Rice Leaf Diseases dataset will be integrated into the FastAPI backend. This will allow the system to extract visual symptom hints (e.g., Brown Spot, Leaf Blast) and pass them as contextual metadata to the RAG retriever.
2. **Expansion of the Dialect Knowledge Graph:** The entity-linking database will be expanded to include broader regional terminologies, improving query recall for farmers across different districts.
3. **Rigorous System Evaluation:** As part of the final system validation, the pipeline will undergo strict quantitative testing. Retrieval performance will be measured using metrics such as Mean Reciprocal Rank (MRR), and translation fidelity will be evaluated using BLEU/ROUGE scores, ensuring the system meets academic and practical deployment standards.

References

1. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459-9474, 2020.
2. S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333-389, 2009.
3. J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535-547, 2019.
4. R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv preprint arXiv:1901.04085, 2019.
5. Microsoft Open Source Blog, "Performant on-device inferencing with ONNX Runtime," 2023. [Online]. Available: https://opensource.microsoft.com
6. A. Dubey et al., "The Llama 3 Herd of Models," arXiv preprint arXiv:2407.21783, 2024.
7. A. Bhattacharjee et al., "BanglaNLG and BanglaT5: Benchmarks and Resources for Evaluating Low-Resource Natural Language Generation in Bangla," Findings of the Association for Computational Linguistics: EMNLP 2022, pp. 7314-7326, 2022.
8. A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision," International Conference on Machine Learning (ICML), pp. 28492-28518, 2023.
9. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019.
10. R. Smith, "An Overview of the Tesseract OCR Engine," Ninth International Conference on Document Analysis and Recognition (ICDAR), pp. 629-633, 2007.
11. LangChain AI, "LangGraph: Multi-Actor, Stateful Applications with LLMs," 2024. [Online]. Available: https://langchain-ai.github.io/langgraph/
12. Kaggle Dataset Repository, "Rice Leaf Diseases Dataset," 2023. [Online]. Available: https://www.kaggle.com/